

Agentic URL Shortener

Interviewer Testing Guide

Runnable .NET 8 prototype - Swagger scenarios - deterministic mode - optional local Ollama mode

Purpose. This document gives a reviewer a repeatable path to run the solution and test the implemented URL-shortener and governed agentic SDLC workflow. It includes exact request samples, expected response patterns, and comments explaining what each scenario demonstrates.

Reviewer note: The agentic workflow produces engineering analysis/proposals. It does not autonomously edit source files, execute generated code changes, or deploy to production. Final completion requires human approval.

Area	Implemented behavior
Runtime	.NET 8 Web API + Swagger
Agentic workflow	Dependency graph, parallel paths, synchronization, context, audit, metrics
Governance	Entry/release gates, retry, fallback, rollback, safe-stop, human approval
LLM mode	Optional Ollama reasoning for RequirementAgent and ArchitectureAgent only
Persistence	In-memory prototype; application restart clears state

1. Prerequisites and Run Steps

Required: .NET 8 SDK and an IDE such as Visual Studio 2022. Ollama is optional and is needed only for the LLM-enabled test section.

Option A - Visual Studio

1. Open **AgenticUrlShortener.sln**.
2. Restore NuGet packages.
3. Build the solution.
4. Run the **AgenticUrlShortener.Api** project using the HTTPS profile.
5. Swagger should open automatically.

Default Swagger URL:

```
https://localhost:7168/swagger
```

Option B - Command line

```
dotnet restore AgenticUrlShortener.sln
dotnet build AgenticUrlShortener.sln
dotnet test AgenticUrlShortener.sln
dotnet run --project src/AgenticUrlShortener.Api
```

Reviewer note: The HTTPS launch profile uses https://localhost:7168 and HTTP uses http://localhost:5168. If the IDE chooses a different profile/port, use the URL printed by ASP.NET Core.

Health check

In Swagger, execute:

```
GET /health
```

Expected response pattern:

```
{
  "status": "healthy",
  "utc": "2026-09-29T..."
}
```

Reviewer note: A 200 response confirms that the API is running. The UTC timestamp changes on every request.

2. Optional Domain API Smoke Test

The repository also contains the working URL-shortener API. This is separate from the agentic SDLC workflow.

Create a short URL

```
POST /api/urls

{
  "url": "https://www.example.com"
}
```

Expected response contains a generated short code and URL metadata. Copy the returned code for the next tests.

Redirect and analytics

```
GET /{code}

GET /api/urls/{code}/analytics
```

Reviewer note: The data store is in memory. If the API is restarted, previously generated short codes no longer exist. Swagger may also show a fetch/CORS-looking message when the redirect endpoint sends the browser to an external website; the redirect can be tested directly in the browser.

3. Test Mode A - LLM Disabled (`Enabled = false`)

Open:

```
src/AgenticUrlShortener.Api/appsettings.json
```

Use:

```
{
  "Llm": {
    "Enabled": false,
    "BaseUrl": "http://localhost:11434",
    "Model": "llama3.2:3b",
    "TimeoutSeconds": 90
  }
}
```

Restart the API after changing configuration. In this mode, all agent outputs come from deterministic C# behavior and Ollama is not required.

Reviewer note: This mode is ideal for repeatable testing and CI. The workflow graph, policy gates, retries, fallback, rollback, audit trail, metrics, replanning, and approval logic are the same in both modes.

Scenario A1 - Greenfield

Goal: demonstrate a new feature requirement flowing through the full SDLC orchestration.

```
POST /api/orchestration/runs
```

```
{
  "scenario": "greenfield",
  "requirement": "Add expiration dates to short URLs."
}
```

Expected high-level result: HTTP 200 with a `WorkflowRun`. All seven steps should complete and the overall status should stop at **WaitingForApproval**.

```
{
  "id": "<generated-guid>",
  "scenario": "greenfield",
  "requirement": "Add expiration dates to short URLs.",
  "status": "WaitingForApproval",
  "planVersion": 1,
  "steps": [
    { "id": "requirement", "agent": "RequirementAgent",
      "status": "Completed", "attempts": 1 },
    { "id": "architecture", "agent": "ArchitectureAgent",
      "status": "Completed", "attempts": 1 },
    { "id": "implementation", "agent": "ImplementationAgent",
      "status": "Completed", "attempts": 1 },
    { "id": "test-design", "agent": "TestingAgent",
      "status": "Completed", "attempts": 1 },
    { "id": "validation", "agent": "ValidationAgent",
      "status": "Completed", "attempts": 1 },
    { "id": "documentation", "agent": "DocumentationAgent",
      "status": "Completed", "attempts": 1 },
    { "id": "release", "agent": "ReleaseReadinessAgent",
      "status": "Completed", "attempts": 1 }
  ],
  "context": { "...": "..." },
  "auditTrail": [ "... events ..." ],
  "completedAt": null
}
```

Reviewer note: Greenfield means a new capability is being planned. The current ImplementationAgent returns an implementation proposal; it does NOT modify ShortUrl.cs or add a real ExpiresAt property. WaitingForApproval is intentional controlled autonomy.

What to inspect in the response

Open `steps` and inspect each `output`. In deterministic mode, RequirementAgent returns a normalized problem/acceptance statement. ArchitectureAgent returns the prototype architecture summary. ImplementationAgent and TestingAgent produce proposals/plans. Validation requires both implementation and test-design outputs before it can complete.

Scenario A2 - Brownfield

Goal: demonstrate reasoning about a change to an existing implementation.

```
POST /api/orchestration/runs

{
  "scenario": "brownfield",
  "requirement": "Update existing analytics to include the last access time."
}
```

Expected status:

```
{
  "scenario": "brownfield",
  "status": "WaitingForApproval",
  "planVersion": 1
}
```

Expected ArchitectureAgent output includes the implemented impact map:

```
Impact map: ShortUrl model -> repository -> UrlShortenerService ->
controllers -> tests. Review current API/data flow before modification;
preserve compatibility.
```

Reviewer note: This scenario demonstrates brownfield/codebase-impact reasoning. The ArchitectureAgent also records the decision to use the smallest compatible change and preserve existing API behavior.

Scenario A3 - Ambiguous Requirement

Goal: demonstrate explicit ambiguity, assumptions, risks, and human review.

```
POST /api/orchestration/runs

{
  "scenario": "ambiguous",
  "requirement": "Make the URL shortener more secure."
}
```

Expected status remains:

```
"status": "WaitingForApproval"
```

Expected RequirementAgent output includes candidate interpretations such as authentication, authorization, rate limiting, URL validation, code entropy, expiry, and abuse controls.

The workflow context should also contain an assumption and risk similar to:

```
assumptions:
- "Security scope is not fully specified; human confirmation is required."

risks:
- "Implementing an unstated security interpretation could solve the wrong problem."
```

Reviewer note: The system does not silently choose one interpretation of 'more secure'. It preserves uncertainty for review. This demonstrates controlled autonomy and decision lineage.

Inspect an existing run

Copy the `id` returned by any workflow and call:

```
GET /api/orchestration/runs/{id}
```

Reviewer note: This endpoint is useful for reviewing step status, attempts, outputs, WorkflowContext, decisions, risks, assumptions, and AuditTrail after the original POST.

4. Human Approval and Recovery Scenarios

Scenario A4 - Human approval

After a successful run reaches WaitingForApproval:

```
POST /api/orchestration/runs/{id}/approve

{
  "approved": true,
  "comment": "Reviewed and approved."
}
```

Expected final status:

```
"status": "Completed"
```

To demonstrate rejection:

```
{
  "approved": false,
  "comment": "Additional changes are required."
}
```

Expected:

```
"status": "Rejected"
```

Reviewer note: Automated agents cannot set the run to Completed. A human decision is mandatory after the release gate.

Scenario A5 - Bounded retry + fallback

```
POST /api/orchestration/runs

{
  "scenario": "greenfield",
  "requirement": "Add expiration dates. [fail-testing]"
}
```

Expected behavior: TestingAgent fails attempt 1, retries once, fails attempt 2, then uses the conservative manual-validation fallback. Its step is changed back to Completed with output beginning:

```
FALLBACK: automated test-design agent failed; use mandatory manual
test checklist and block release until human review.
```

The context also records a lower-confidence testing risk. The workflow can continue to WaitingForApproval.

Reviewer note: This scenario demonstrates the difference between retry and fallback: retry repeats the same automated operation; fallback substitutes a safer alternative after retries are exhausted.

Scenario A6 - Rollback

```
POST /api/orchestration/runs

{
  "scenario": "greenfield",
  "requirement": "Add expiration dates. [fail-implementation]"
}
```

Expected overall status:

```
"status": "RolledBack"
```

Reviewer note: ImplementationAgent intentionally fails twice. No safe implementation fallback exists, so previously completed automated workflow steps are marked RolledBack. This is workflow-state rollback, not Git or production rollback.

Scenario A7 - Safe-stop policy

```
POST /api/orchestration/runs

{
  "scenario": "greenfield",
```

```
"requirement": "Deploy directly to production."  
}
```

Expected overall status:

```
"status": "SafeStopped"
```

Reviewer note: The entry policy gate blocks direct production deployment before normal agent execution. Safe-stop means the system halts rather than proceeding with an unsafe/invalid request.

5. Dynamic Replanning and Metrics

Scenario A8 - Replan after a requirement change

Start a normal run, copy its `id`, then call:

```
POST /api/orchestration/runs/{id}/replan

{
  "updatedRequirement":
    "Add expiration dates and a configurable grace period.",
  "reason":
    "Requirement changed after architecture review."
}
```

Expected response pattern:

```
{
  "id": "<new-run-guid>",
  "requirement":
    "Add expiration dates and a configurable grace period.",
  "planVersion": 2,
  "status": "WaitingForApproval",
  "context": {
    "values": {
      "parentRunId": "<original-run-guid>",
      "previousRequirement": "Add expiration dates to short URLs.",
      "...": "..."
    },
    "decisions": [
      "...",
      "Replanned because: Requirement changed after architecture review."
    ]
  }
}
```

Reviewer note: The original run is preserved and marked SafeStopped/superseded. The new run keeps parent-run lineage and increments planVersion. This demonstrates governed replanning rather than overwriting history.

Reliability metrics

```
GET /api/orchestration/metrics
```

Expected fields:

```
{
  "totalRuns": 0,
  "successfulRuns": 0,
  "failedOrStoppedRuns": 0,
  "retryCount": 0,
  "rollbackCount": 0,
  "replanCount": 0,
  "averageLatencyMs": 0,
  "averageRecoveryTimeMs": 0,
  "successRate": 0
}
```

Reviewer note: The numeric values depend on what has been executed in the current process. Metrics are in memory and reset when the API restarts.

6. Test Mode B - LLM Enabled (`Enabled = true`)

This optional mode uses a local Ollama model. Only **RequirementAgent** and **ArchitectureAgent** call the LLM. Implementation, Testing, Validation, Documentation, ReleaseReadiness, policies, state transitions, retries, rollback, safe-stop, and approval remain deterministic.

Prepare Ollama

```
ollama pull llama3.2:3b
ollama list
```

Ensure the local Ollama server is running. Then change:

```
{
  "Llm": {
    "Enabled": true,
    "BaseUrl": "http://localhost:11434",
    "Model": "llama3.2:3b",
    "TimeoutSeconds": 90
  }
}
```

Restart the .NET API.

Reviewer note: The first local-model request may be slower because the model may need to load into memory. If Ollama is unavailable or times out, the agent call fails and the orchestrator's retry/recovery logic applies; the system does not pretend an LLM ran.

Scenario B1 - Ambiguous requirement with LLM reasoning

```
POST /api/orchestration/runs
```

```
{
  "scenario": "ambiguous",
  "requirement": "Make the URL shortener more secure."
}
```

Expected overall structure is the same as deterministic mode and, when the LLM calls succeed, the final status should still be:

```
"status": "WaitingForApproval"
```

The key difference is the `output` of RequirementAgent and ArchitectureAgent. Each should contain both the deterministic baseline and an LLM section:

```
{
  "id": "requirement",
  "agent": "RequirementAgent",
  "status": "Completed",
  "attempts": 1,
  "output":
    "BASELINE: <deterministic requirement analysis>\n
    LLM ANALYSIS (llama3.2:3b): <model-generated analysis>"
}

{
  "id": "architecture",
  "agent": "ArchitectureAgent",
  "status": "Completed",
  "attempts": 1,
  "output":
    "BASELINE: <deterministic architecture constraints>\n
    LLM ANALYSIS (llama3.2:3b): <model-generated analysis>"
}
```

Reviewer note: Exact LLM wording is intentionally not fixed. The model runs with temperature 0.7, so two valid runs can phrase the analysis differently. The reviewer should verify the presence of 'LLM ANALYSIS (llama3.2:3b)' rather than compare exact prose.

Scenario B2 - Greenfield with LLM reasoning

POST /api/orchestration/runs

```
{  
  "scenario": "greenfield",  
  "requirement": "Add expiration dates to short URLs."  
}
```

Expected: RequirementAgent and ArchitectureAgent contain `BASELINE` + `LLM ANALYSIS`; the remaining agents use deterministic outputs; the successful workflow stops at WaitingForApproval.

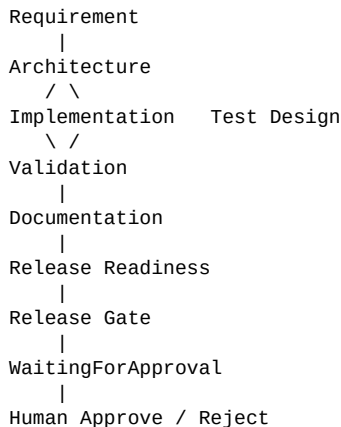
Reviewer note: LLM mode changes how two reasoning-heavy stages generate analysis. It does not change the workflow graph or give the model authority to edit files, bypass policy, approve a release, or deploy.

7. Recommended Reviewer Sequence

#	Test	Expected proof
1	GET /health	API is running
2	dotnet test	Automated unit/integration suite executes
3	Enabled=false greenfield	Full deterministic SDLC path -> WaitingForApproval
4	Brownfield	Existing-code impact map and compatibility decision
5	Ambiguous	Assumptions/risks preserved for review
6	Approve a run	WaitingForApproval -> Completed
7	[fail-testing]	Two attempts -> manual-validation fallback
8	[fail-implementation]	Two attempts -> RolledBack
9	Direct production request	Entry gate -> SafeStopped
10	Replan	New run, planVersion increment, parent lineage
11	GET metrics	Retries/rollbacks/replans/latency observability
12	Enabled=true + Ollama	LLM analysis only in Requirement/Architecture

8. What the Workflow Demonstrates

The orchestration is not a simple linear chain. Requirement runs first, followed by Architecture. Implementation and Test Design then become ready together and execute through `Task.WhenAll`. Validation is the synchronization point and cannot run until both are complete. Documentation and Release Readiness follow, then the release policy gate stops at WaitingForApproval.



Context and lineage: WorkflowContext preserves values, decisions, risks, and assumptions. AuditTrail records workflow/step transitions. Replanning preserves parentRunId and previousRequirement.

Controlled autonomy: agents can perform bounded workflow work, but policy gates and the orchestrator remain authoritative. A human owns the final approval decision.

9. Prototype Limitations

- URL, workflow, and metrics data are stored in memory; restart loses state.
- ImplementationAgent produces an implementation proposal; it does not edit source files.
- TestingAgent produces a validation plan; it does not execute newly generated tests.
- Rollback invalidates workflow artifacts/state; it is not a Git/deployment rollback.
- LLM mode is optional and limited to RequirementAgent and ArchitectureAgent.

- Ollama integration is local and does not include production authentication/tool execution.
- Policy rules are illustrative prototype guardrails, not an enterprise policy engine.

Reviewer note: These limitations are intentional boundaries of the take-home prototype and should be considered when evaluating what is demonstrated versus what would be added for production.

10. Expected Successful Review Outcome

A successful review should show that the prototype runs end-to-end, exposes a working URL-shortener API, executes a stateful dependency graph, supports parallel work and synchronization, preserves context and audit history, applies policy gates, demonstrates bounded recovery behaviors, supports dynamic replanning, records reliability metrics, and requires human approval before final completion. Optional LLM mode adds local model reasoning without transferring governance authority to the model.